

Microservices Data Management – Choosing Right Database

Introduction

La struttura delle tre lezioni sui dati

*"La gestione dei dati nei microservizi è un argomento così ampio e importante che suddivideremo la nostra esplorazione in **tre aree principali**:"*

Area	Lezione
1. Scegliere il database giusto e le strategie di persistenza	Lezione 10 (questa)
2. Gestire comandi e query	Lezione 11
3. Gestire le transazioni distribuite	Lezione 12

Il problema ereditato

*"I database creano colli di bottiglia quando si scala. I database sono più difficili da scalare rispetto ai servizi applicativi stateless, e lo **scaling verticale** ha limiti e costi elevati."*

La roadmap dei pattern

Pattern	Dove
Database-per-Service (fondamentale)	Lez. 10
Shared Database (⚠ anti-pattern da evitare)	Lez. 10
API Composition / Gateway Aggregation per le query	Lez. 11
CQRS — separare letture e scritture	Lez. 11
Event Sourcing — catturare i cambiamenti come sequenza di eventi	Lez. 11
Saga — transazioni distribuite	Lez. 12

*"Tutti questi pattern affrontano **le sfide dei database relazionali ACID monolitici** rispetto alle esigenze di dati e scalabilità delle applicazioni."*

The Database-per-Service Pattern

(Già introdotto nella Lezione 5; qui viene ripreso e approfondito con le sfide.)

Definizione

Ogni microservizio possiede e gestisce il proprio database privato, dedicato esclusivamente a quel servizio, che memorizza solo i dati rilevanti per la sua specifica capability di business.

*"Secondo questo pattern, **il data store del servizio è considerato parte della sua implementazione privata. Nessun altro servizio può accedervi direttamente. Non c'è condivisione di tabelle né query dirette incrociate.**"*

*"Se un microservizio ha bisogno di dati di proprietà di un altro, **deve ottenerli chiamando l'API pubblica esposta dal servizio proprietario.**"*

Gli esempi del corso:

Microservizio	Database	Perché
Catalog	MongoDB (documentale NoSQL)	"Dati di prodotto flessibili e letture elevate "
Basket	Redis (key-value)	"Dati temporanei e veloci del carrello"
Ordering	SQL Server (relazionale)	"Dati transazionali degli ordini, che richiedono forte coerenza transazionale "

*"Il microservizio Catalog potrebbe avere **un carico di lettura elevato**, mentre Ordering **un carico di scrittura elevato durante i picchi di vendita**. In base a questi requisiti possiamo progettare i database seguendo il pattern **polyglot persistence**."*

★ Le 4 sfide

1. Nessun join diretto tra database di servizi — "questa è una cosa importante"

*"Se devi eseguire una query che combina dati di **Orders DB** e **Products DB**, non è più possibile scrivere un semplice **join SQL** tra tabelle di database diversi."*

2. Le transazioni distribuite diventano complesse

*"Come garantire l'**atomicità** di un'operazione di business che richiede l'aggiornamento di dati in più servizi, ciascuno col proprio database? **Le transazioni ACID tradizionali che si estendono su più database in un sistema distribuito sono molto difficili e spesso sconsigliate.** Questo porta alla necessità di pattern come le **Saga**."*

3. Aumento dei costi operativi

*"Gestire più istanze di database, **potenzialmente di tecnologie diverse** a causa della polyglot persistence, aumenta la complessità operativa: deployment, monitoring, backup e manutenzione."*

4. Dipendenza dalla comunicazione inter-servizio

*"I servizi devono usare le API per recuperare i dati altrui. **Questo introduce latenza di rete** o richiede **pattern di comunicazione resilienti come retry e circuit breaker.**"*

The Shared Database Anti-Pattern & Polyglot Persistence

L'anti-pattern

*"Se **non** si segue il Database-per-Service e si usa **un database condiviso** per diversi microservizi, è **diventato un anti-pattern.**"*

*"È possibile creare un singolo database condiviso, accedendo ai dati tramite **transazioni ACID locali**. Ma è **contrario alla natura dei microservizi** e causerà seri problemi."*

★ *"Quando si usa un database condiviso, i microservizi perdono le loro proprietà fondamentali: scalabilità, resilienza e indipendenza. Alla fine dei conti, dovrete sviluppare grandi applicazioni monolitiche invece di microservizi."*

I 5 danni concreti

Danno	Descrizione
Le modifiche allo schema rompono altri servizi	"Se una modifica allo schema condiviso viene apportata da un servizio, può rompere l'altro "
Perdita di indipendenza	"I team devono coordinare le modifiche al database , rallentando i rilasci. Modificare il database diventa un evento importante "
Collo di bottiglia centrale	"Limita la scalabilità indipendente dei servizi"
Niente polyglot persistence	" Tutti i servizi sono legati alla stessa tecnologia di database "
Single point of failure	"Se il database condiviso ha problemi, più servizi ne risentono "

★ La domanda onesta

*"Se il database condiviso sembra essere l'opzione migliore, **dovremmo ripensarci: abbiamo davvero bisogno di microservizi? Forse l'architettura monolitica sarebbe la scelta migliore, e dovremmo considerare l'approccio modular monolith invece dei microservizi.**"*

Questa è una delle osservazioni più mature del corso: **se non puoi separare i dati, non stai facendo microservizi — stai facendo un monolite distribuito**, e il modular monolith della Lezione 3 sarebbe una scelta migliore e più onesta.

I "vantaggi" del database condiviso (perché a volte lo si usa)

- **Gestione delle transazioni semplice** — *"non richiede di distribuire le transazioni sui microservizi"*
- **Riduzione dei dati duplicati**
- **Query complesse con join facili**, poiché i dati sono tutti insieme
- **Principi ACID e coerenza dei dati facilmente gestibili**

Ma: *"Per questo è meglio seguire un'architettura modular monolith."*

Polyglot Persistence — il richiamo a Martin Fowler

*"**Martin Fowler** ha un ottimo articolo sul principio della polyglot persistence, e spiega che la polyglot persistence avrà un costo, ma ne vale la pena. Quando i database relazionali vengono usati in modo inappropriato, danneggiano lo sviluppo delle applicazioni."*

L'esempio:

*"Se hai solo ricerche per ID, e non hai bisogno di transazioni né di condividere il database, **non ha senso usare un database relazionale**. Un problema come questo è molto più adatto a un database NoSQL key-value."*

Relational and NoSQL Databases

Database relazionali

Caratteristiche:

- Dati organizzati in **tabelle** (righe e colonne)
- **Schema fisso**, definito in anticipo
- Interrogati con **SQL**
- Relazioni esplicite tramite **chiavi primarie e chiavi esterne**
- ★ **Forte supporto alle transazioni ACID**

Definizione: ACID — Atomicity, Consistency, Isolation, Durability. *"Se una parte di una transazione multi-step fallisce, l'intera transazione viene annullata, mantenendo la coerenza dei dati."*

Quando è un'eccellente scelta: *"per microservizi i cui dati sono **altamente strutturati**, richiedono **forte coerenza transazionale**, o beneficiano di **query relazionali complesse**."*

Esempi: PostgreSQL, MySQL, SQL Server, Oracle.

Database NoSQL

*"NoSQL non significa "no SQL" ma "**not only SQL**" o "non relazionale". È un termine generico per un'ampia categoria di sistemi che differiscono per **come memorizzano i dati**, **gestiscono gli schemi**, e **quali caratteristiche privilegiano**."*

Gli aspetti chiave:

- Progettati per **modelli di dati oltre le tabelle rigide**: documenti, coppie chiave-valore, wide-column, grafi
- **Schema-less o schemi molto flessibili** → più facile evoluzione delle strutture dati
- **Costruiti da zero per la scalabilità orizzontale**, alte prestazioni e alta disponibilità
- ★ *"Mentre i relazionali enfatizzano la **forte coerenza ACID**, molti NoSQL operano con **modelli di coerenza diversi**, spesso propendendo per l'**eventual consistency** a favore di **maggior disponibilità e tolleranza alle partizioni**."*

Definizione: BASE — l'alternativa NoSQL ad ACID: **B**asically **A**vailable, **S**oft state, **E**ventual consistency.

★ I 4 tipi di database NoSQL

1. Document Database

Memorizzano i dati in documenti, tipicamente in JSON o BSON (JSON binario).

- *"Ogni documento è un'unità di informazione autonoma, che spesso rappresenta un oggetto dell'applicazione"*
- *"Un prodotto con tutti i suoi attributi, e persino oggetti annidati come specifiche o recensioni, può essere memorizzato come un unico documento JSON"*
- ★ **Flessibilità dello schema:** *"documenti diversi della stessa collezione possono avere strutture o campi diversi"*
- *"Poiché i dati correlati sono spesso incorporati (embedded) nel documento, questo modello riduce la necessità di join complessi"*

Ideali per: CMS, **cataloghi di prodotti** ("dove gli attributi variano notevolmente"), profili utente, dati gerarchici complessi.

Esempi: MongoDB, Couchbase, Amazon DocumentDB.

*"Per il nostro servizio **Catalog**, un database documentale potrebbe essere un'ottima soluzione."*

2. Key-Value Store

Il tipo più semplice di NoSQL. I dati sono memorizzati come coppie chiave-valore.

- "Ogni dato ha **una chiave univoca**, e a questa è associato un valore"
- Il valore può essere semplice (stringa, numero) o complesso (**oggetto JSON serializzato**)
- ★ **"Il modo principale per accedere ai dati è la chiave, il che li rende estremamente veloci nelle ricerche"**

Ideali per: **caching** ("come abbiamo fatto con l'output cache con Redis"), **gestione delle sessioni utente**, qualsiasi applicazione con ricerche velocissime per chiave nota.

Esempi: Redis, Memcached, Amazon DynamoDB, Azure Cosmos DB.

*"Potremmo usarlo per i dati del nostro **servizio Basket**."*

3. Wide-Column Store (Column-Family)

Memorizzano i dati in tabelle, righe e colonne, ma con una differenza: l'insieme delle colonne può variare da riga a riga, e le colonne sono raggruppate in "column family".

- ★ **"Sono altamente ottimizzati per query che accedono solo a un sottoinsieme di colonne su un numero potenzialmente enorme di righe. Leggere colonne specifiche è molto efficiente, perché il database non deve leggere intere righe"**
- **"Progettati per estrema scalabilità orizzontale e alta disponibilità, in particolare per carichi di scrittura pesanti e big data"**
- **"Colonne e column family possono essere distribuite su diversi server"**

Ideali per: data warehousing e big data, **dati time-series** (letture di sensori, log applicativi), motori di raccomandazione, sistemi con **throughput di scrittura molto elevato**.

Esempi: Apache Cassandra, Apache HBase, Google Bigtable, Azure Cosmos DB (Cassandra API).

4. Graph Database

Progettati per memorizzare e navigare dati il cui valore risiede nelle RELAZIONI.

- I dati sono modellati come un **grafo di nodi e archi (edge)**
- **Nodi** = entità (cliente, prodotto, ordine)
- **Archi** = relazioni ("cliente **ha acquistato** prodotto", "utente è **amico di** utente")
- **"Sia i nodi che gli archi possono avere proprietà e attributi"**

★ **Il punto di forza:**

"La capacità di **interrogare e attraversare efficientemente relazioni complesse**. Ad esempio, **trovare gli amici degli amici che hanno acquistato prodotti simili** è molto più **veloce e naturale** in un graph database che fare molti join complessi in un relazionale."

Ideali per: social network, **motori di raccomandazione**, **rilevamento frodi** ("identificare schemi sospetti di connessioni"), knowledge graph, analisi di rete.

Esempi: Neo4j, Amazon Neptune, Azure Cosmos DB (Gremlin API).

When to Use Relational or NoSQL?

Usa RELAZIONALE quando...

Motivo	Dettaglio
Serve conformità ACID e forte coerenza	"Se il microservizio gestisce transazioni finanziarie o processi dove l'integrità e la coerenza immediata sono cruciali "
I dati sono altamente strutturati e prevedibili	"Schema relativamente stabile e relazioni chiare tra le entità"
Servono query e join complessi	"Se il servizio deve generare report complessi o eseguire analisi sofisticate. SQL è difficile da battere "
La normalizzazione è un obiettivo chiave	"Riduce ridondanza e anomalie "
Volumi moderati o prevedibili	"Se il volume rientra comodamente nelle capacità di un singolo server scalato verticalmente"
<i>"Il microservizio Ordering, con le sue esigenze di integrità transazionale e dati strutturati, è un candidato classico per PostgreSQL o SQL Server."</i>	

✓ Usa NoSQL quando...

Motivo	Dettaglio
Serve schema flessibile e dati dinamici ★	"È possibile aggiungere o rimuovere attributi senza complesse migrazioni di schema . Se la struttura cambia frequentemente o varia molto tra elementi — attributi di prodotti in un catalogo diversificato, contenuti generati dagli utenti, una piattaforma IoT che ingerisce dati da sensori diversi "
Serve alta scalabilità orizzontale e throughput	"Progettati per scalare su cluster di server commodity . Se devi gestire milioni di operazioni al secondo o dataset che superano la capacità di un singolo server "
Alta disponibilità e tolleranza alle partizioni sono fondamentali	"Privilegiano A e P del teorema CAP , spesso allentando la forte coerenza a favore dell'eventual consistency "
Il modello dati si adatta naturalmente	Oggetti complessi autonomi → documentale ; ricerche per chiave → key-value ; dati altamente connessi → graph ; dataset massicci write-heavy → wide-column
Accesso semplice e denormalizzazione accettabile	"Molti NoSQL ottengono prestazioni memorizzando dati denormalizzati, riducendo o eliminando i join "

How to Choose a Database + CAP Theorem

Le best practice di selezione

1. "Lo strumento giusto per il lavoro giusto"

"Non dovremmo cercare di costringere un database relazionale a fare tutto. I microservizi offrono la libertà di usare un mix di tecnologie di data store."

2. Tipo di dati e pattern di accesso

Natura dei dati	Scelta
Altamente strutturati, relazioni complesse	Relazionale SQL
Documenti JSON con schemi flessibili	Document DB
Semplice lookup per chiave (cache, sessioni)	Key-value
Telemetria o log	Time-series DB o Elasticsearch

3. Trade-off disponibilità vs coerenza — "questo è un punto critico"

4. Allineamento con i confini transazionali — "porta al pattern Saga"

5. Competenze del team

*"Sebbene sia bene usare lo strumento migliore, **va considerata anche la capacità del team di apprendere, gestire, ottimizzare le query e fare tuning per qualsiasi nuova tecnologia introdotta.**"*

Le 8 domande guida per ogni microservizio

1. **Che livello di coerenza serve?** Forte ACID (transazioni finanziarie) o **eventual** accettabile (feed, raccomandazioni)?
2. **Quanto deve essere flessibile lo schema?** Prevedibile e stabile → relazionale. Dinamico → NoSQL.
3. **Volume dati e caratteristiche del carico?** Alto volume → NoSQL. **Read-heavy, write-heavy o bilanciato?**
4. **Complessità delle query?** Join complessi su dati normalizzati → SQL. Lookup per chiave o query su documenti denormalizzati → **NoSQL**.
5. **Struttura di distribuzione desiderata?** Centralizzata o **replicata su zone geografiche?**
6. **Requisiti di performance rigorosi?** Serve **latenza estremamente bassa?**
7. **Requisiti di scalabilità?** **Scalabilità orizzontale** è l'obiettivo principale?
8. **Requisiti di disponibilità?** Quanto è importante che il data store sia **sempre disponibile?**

★ Il Teorema CAP

La domanda che lo introduce:

"Possiamo trovare una tecnologia di database che ci dia massima scalabilità, 100% di disponibilità, prestazioni fulminee, query relazionali complesse E garantisca forte coerenza in un sistema distribuito?"

"Purtroppo la risposta è no. Non esiste un database silver bullet."

Definizione:

*"Il teorema CAP, proposto da **Eric Brewer**, afferma che **in qualsiasi data store distribuito è impossibile fornire simultaneamente più di due** di queste tre garanzie."*

(Nota: Brewer formulò la congettura nel 2000; fu dimostrata formalmente da Gilbert e Lynch nel 2002. La trascrizione dice 1998.)

Le tre garanzie

C — Consistency (coerenza forte)

*"**Tutti i nodi vedono gli stessi dati nello stesso momento. Ogni lettura riceve la scrittura più recente, o un errore. Se una scrittura avviene su un nodo, tutte le letture successive da qualsiasi nodo devono riflettere quella scrittura. Se questo non può essere garantito, il sistema potrebbe dover bloccare le richieste o restituire un errore.**"*

A — Availability (disponibilità)

*"**Ogni richiesta riceve una risposta, non un errore. Sebbene non sia garantito che la risposta contenga la scrittura più aggiornata, il sistema è sempre operativo e in grado di rispondere, anche se alcuni nodi sono inattivi o irraggiungibili.**"*

P — Partition Tolerance (tolleranza alle partizioni)

*"**La capacità del sistema di continuare a funzionare anche se si verifica una partizione di rete — ovvero se la comunicazione tra alcuni nodi viene persa o ritardata. I nodi potrebbero essere attivi ma non in grado di parlarsi.**"*

★ La vera scelta: CP o AP

*"**Nel mondo reale, le partizioni di rete sono una realtà inevitabile. Le reti non sono perfette, i messaggi possono essere persi o ritardati. Pertanto, un database distribuito DEVE tollerare le partizioni per essere robusto.**"*

*"Questo significa che, **quando si verifica una partizione, si è costretti a scegliere tra coerenza e disponibilità.**"*

Quindi P **non è opzionale** in un sistema distribuito. La scelta reale è tra:

	Sistemi CP (Consistency + Partition tolerance)	Sistemi AP (Availability + Partition tolerance)
Priorità	Coerenza sopra disponibilità	Disponibilità sopra coerenza
Durante una partizione	<i>"Alcune parti del sistema diventano non disponibili, per evitare il rischio che il client legga o scriva dati contrastanti o non aggiornati"</i>	<i>"Tutti i nodi sani continuano ad accettare letture e scritture"</i>
Il costo	Downtime parziale	<i>"Servire potenzialmente dati obsoleti (stale), o dover risolvere in seguito scritture conflittuali"</i>
Dopo la partizione	<i>"Ristabilita la coerenza, viene ripristinata la piena disponibilità"</i>	Si applica la riconciliazione
Esempi	Alcune configurazioni cluster di RDBMS tradizionali	Cassandra, Amazon DynamoDB, Couchbase

★ *"La maggior parte delle applicazioni web su larga scala e distribuite globalmente, e delle architetture a microservizi, tende a orientarsi verso i sistemi AP, privilegiando l'alta disponibilità e gestendo la coerenza tramite modelli di eventual consistency."*

"Perché per molti casi d'uso, essere sempre disponibili è più critico che garantire coerenza istantanea assoluta al costo di potenziali downtime."

Un esempio che rende concreta la scelta: se il carrello di un e-commerce mostra un prodotto con un prezzo aggiornato 2 secondi fa invece di adesso, nessuno muore (AP va bene). Se un bonifico bancario viene eseguito due volte perché due nodi non si parlavano, è un disastro (serve CP).

Problem: Single Database Server Performs Low Performance

Lo stato attuale

*"Abbiamo implementato correttamente il **Database-per-Service**: Catalog ha il suo Products DB, Ordering il suo Orders DB. Questo dà eccellente separazione logica e autonomia."*

*"Tuttavia, se ognuno di questi database gira su **una singola istanza del server**, quella singola istanza — per quanto potente sia stata resa con lo scaling verticale — può alla fine diventare un collo di bottiglia."*

Le domande che aprono il problema

"Cosa succede quando il volume di dati del Catalog DB diventa enorme? O quando il tasso di transazioni dell'Orders DB diventa eccessivo durante il Black Friday, anche per un singolo server molto grande?"

I 5 sintomi

1. **Query più lente** — "quando le tabelle crescono, le query su grandi quantità di dati diventano più lente"
2. **Esaurimento risorse** — "un elevato carico di richieste simultanee può **sovraccaricare CPU, memoria e I/O**, aumentando la latenza per tutte le operazioni"
3. **Esaurimento dello spazio disco** — "con l'accumularsi dei dati"
4. **Manutenzione lenta e rischiosa** — "backup, ripristino o manutenzione su un singolo database di grandi dimensioni diventano **sempre più lunghi e rischiosi**"
5. **Single point of failure** — "se la singola istanza si guasta, **l'intero microservizio che vi si basa diventa indisponibile**"

L'obiettivo

*"Distribuire i dati del servizio su **più server di database** che possano essere gestiti e scalati in modo più indipendente."*

I benefici attesi:

- **Aumento delle prestazioni delle query** lavorando con **sottoinsiemi di dati più piccoli**
- **Aumento di scalabilità e disponibilità** del livello dati

Data Partitioning: Horizontal, Vertical, Functional

Definizione

Il partizionamento dei dati è il processo di prendere un database o un dataset di grandi dimensioni e dividerlo in segmenti più piccoli, più gestibili e spesso accessibili in modo indipendente.

"I sistemi di database spesso forniscono un'astrazione, così che le applicazioni client interagiscano con quello che sembra un singolo database logico. L'idea principale è distribuire i dati tra queste partizioni, che possono risiedere su dispositivi di storage diversi o addirittura su server diversi."

I 5 vantaggi

Vantaggio	Dettaglio
Scalabilità ★	"Il fattore principale. Suddividendo i dati su server separati, si supera il limite di una singola macchina"
Maggiore disponibilità	"Se un server che gestisce una partizione si guasta, solo quei dati sono temporaneamente non disponibili. Gli altri restano accessibili"
Migliori prestazioni	"Le query possono essere molto più veloci, perché scansionano solo una partizione più piccola e rilevante invece di un'intera tabella enorme. Inoltre, alcune operazioni possono essere parallelizzate tra le partizioni"
Migliore gestibilità	"Backup, ricostruzione degli indici e manutenzione sono più efficienti su pezzi più piccoli"
Sicurezza	"Si possono archiviare dati altamente sensibili in partizioni specifiche, con controlli di accesso rigorosi o crittografia"

★ Le 3 strategie

1. Partizionamento ORIZZONTALE (Sharding)

Si prende una tabella grande e la si divide in più tabelle più piccole (shard), distribuendo le RIGHE.

- ★ Ogni shard ha lo STESSO SCHEMA (le stesse colonne), ma contiene un sottoinsieme distinto di righe



Definizione: partition key (chiave di partizione)

"Una colonna specifica o un insieme di colonne il cui valore viene usato per decidere a quale shard appartiene una determinata riga."

Esempi: ID cliente, ID prodotto, area geografica, intervallo di date.

"Quando un'applicazione deve interrogare o aggiornare i dati, il livello di routing usa il valore della partition key per individuare lo shard specifico."

⚠ La sfida principale — gli hotspot:

"Una chiave scelta male può portare a una distribuzione non uniforme dei dati, con alcuni shard che diventano "hotspot" — gestendo una quantità sproporzionata di dati mentre altri sono sottoutilizzati, riducendo i vantaggi dello sharding."

2. Partizionamento VERTICALE

Si prende una singola tabella e si dividono le sue COLONNE in più tabelle. Ogni nuova tabella contiene un sottoinsieme delle colonne originali, ma tipicamente TUTTE le righe.

"Queste nuove tabelle sono collegate usando la chiave primaria della tabella originale."

Come si dividono le colonne:

- Colonne ad accesso frequente in una partizione, colonne ad accesso raro in un'altra
- Colonne che cambiano raramente separate da quelle aggiornate di frequente

Esempio — social media:



Il vantaggio:

"Migliora le prestazioni delle query se queste accedono tipicamente solo a un sottoinsieme di colonne. Leggendo solo le colonne necessarie da una tabella più stretta, l'I/O si riduce."

3. Partizionamento FUNZIONALE

I dati vengono suddivisi e raggruppati in base al loro utilizzo da parte delle diverse funzioni di business, bounded context DDD, o caratteristiche distinte dell'applicazione.

★ *"E questo si allinea molto bene al modo in cui decomponiamo l'applicazione in microservizi quando applichiamo il Database-per-Service. Sebbene ogni microservizio possieda i propri dati, stiamo essenzialmente implementando una forma di partizionamento funzionale ad alto livello."*

Nel nostro e-shop:

Partizione funzionale	Dati	Gestita da
Catalogo	prodotti, categorie, specifiche	Catalog Service
Ordini	ordini, articoli, pagamenti, spedizione	Ordering Service
Clienti	account cliente	Identity Service

*"Il partizionamento funzionale **crea confini chiari di proprietà dei dati** e consente alle diverse parti dell'applicazione di **evolvere i propri modelli dati e scalare i propri data store in modo indipendente**. Si traduce spesso nell'uso di **tecnologie di database diverse** per le diverse partizioni funzionali — **portando alla polyglot persistence**."*

In sintesi: il partizionamento funzionale è **ciò che abbiamo già fatto** nelle Lezioni 5-6 (asse Y). Il partizionamento orizzontale/sharding è **quello che ci manca** (asse Z).

Database Sharding Pattern & Tinder Example

Definizione

Lo sharding è una tecnica per partizionare orizzontalmente un database: dividere un database grande in pezzi indipendenti più piccoli, più veloci e più gestibili, chiamati shard o chunk.

"Ogni shard è essenzialmente una propria istanza di database, che contiene un sottoinsieme distinto delle righe totali. Tutti gli shard hanno lo stesso schema — le stesse tabelle e colonne."

La shard key:

*"Una o più colonne — **ID utente, ID sede, ID tenant** — il cui valore viene usato **da una logica di sharding o da un livello di routing** per determinare su quale shard risiede un particolare dato."*

I 3 motivi per implementarlo

1. Scalabilità massiccia

*"Quando il volume o il carico cresce **oltre la capacità di un singolo server**, lo sharding consente di **scalare aggiungendo altri shard**, ognuno tipicamente sul proprio server."*

2. Maggiore disponibilità e isolamento dai guasti

"Il guasto di un server non comporta necessariamente il blocco dell'intero database. I dati negli altri shard rimangono accessibili."

3. Distribuzione geografica (Geo-Sharding) ★

*"Gli shard possono essere fisicamente situati più vicino agli utenti che servono. I dati degli utenti europei nel datacenter europeo, quelli USA nel datacenter USA. **Riduce la latenza di rete** e può contribuire alla **conformità normativa sui dati** (data residency)."*

★ L'esempio Tinder

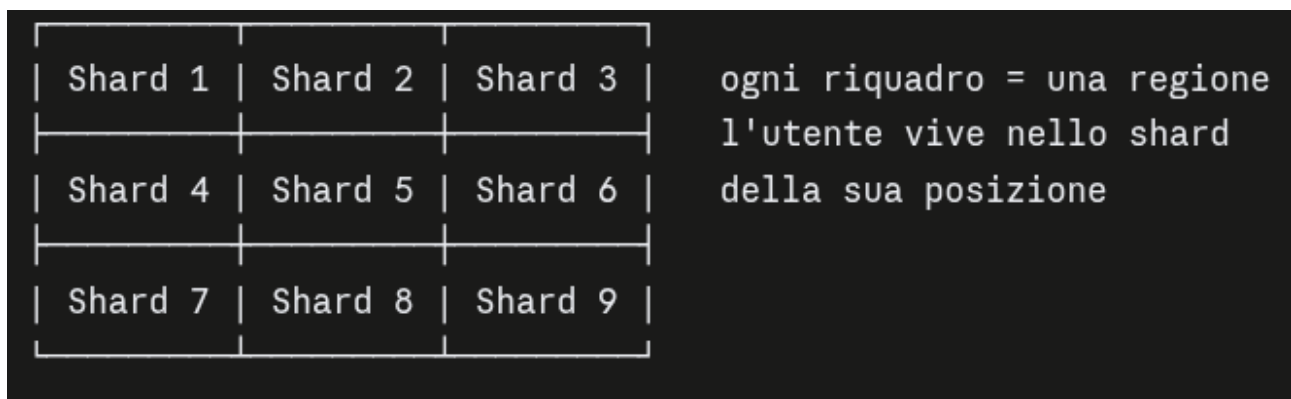
Il problema:

*"La funzionalità principale di Tinder consiste nell'**abbinare utenti geograficamente vicini** — tipicamente entro un certo raggio, ad esempio **160 km**. Deve farlo **molto rapidamente per milioni di utenti attivi**."*

"Immaginate di cercare corrispondenze vicine se tutti i profili globali fossero in un unico enorme database: le query basate sulla posizione sarebbero incredibilmente lente e inefficienti."

La soluzione — Geo-Sharding:

"Tinder segmenta il suo enorme database utenti in base alla posizione geografica. Concettualmente, la mappa del mondo è divisa in numerose regioni o caselle distinte. Ciascuna regione rappresenta uno shard. I dati del profilo di un utente sono memorizzati nello shard che corrisponde alla sua posizione geografica."



"Quando un utente apre l'app e inizia a scorrere, invece di cercare nell'intera base utenti globale, l'applicazione interroga principalmente gli shard pertinenti alla sua posizione attuale e al raggio di ricerca."

I benefici:

"Drastica riduzione della portata della ricerca, minore latenza, e scalabilità, perché le query scansionano solo un sottoinsieme molto più piccolo e geograficamente rilevante dei dati totali."

★ *"Questo è un esempio perfetto di come la scelta della giusta strategia di sharding, basata sulla funzionalità principale dell'applicazione, possa portare a un sistema altamente performante e scalabile."*

Apache Cassandra

"Perché stiamo imparando Cassandra adesso? Perché è un ottimo esempio di implementazione del pattern di sharding e di scalabilità orizzontale."

Definizione:

"Un database distribuito NoSQL wide-column della Apache Foundation, altamente scalabile e ad alte prestazioni, progettato per gestire grandi quantità di dati strutturati su diversi server, garantendo alta disponibilità senza un singolo punto di guasto."

Caratteristiche:

- **Scalabilità elastica** (aggiungi nodi → aumenta capacità)
- **Nessun single point of failure**
- Orientato alle colonne, **archiviazione flessibile**
- **Distribuzione dei dati facile**

★ L'architettura peer-to-peer masterless:

"Cassandra ha un sistema distribuito peer-to-peer tra i nodi di un cluster. Ogni nodo Cassandra ha lo stesso ruolo e funziona come master. Possiamo definirla un'architettura masterless (senza master)."

Perché conta: in un'architettura master-slave, il master è un single point of failure e un collo di bottiglia per le scritture. In una masterless, **qualsiasi nodo può accettare qualsiasi richiesta**, e i dati vengono distribuiti automaticamente tramite consistent hashing (ricordi la Lezione 4?). È la ragione per cui Cassandra è un classico sistema **AP**.

Design: Microservices Architecture with Database Sharding

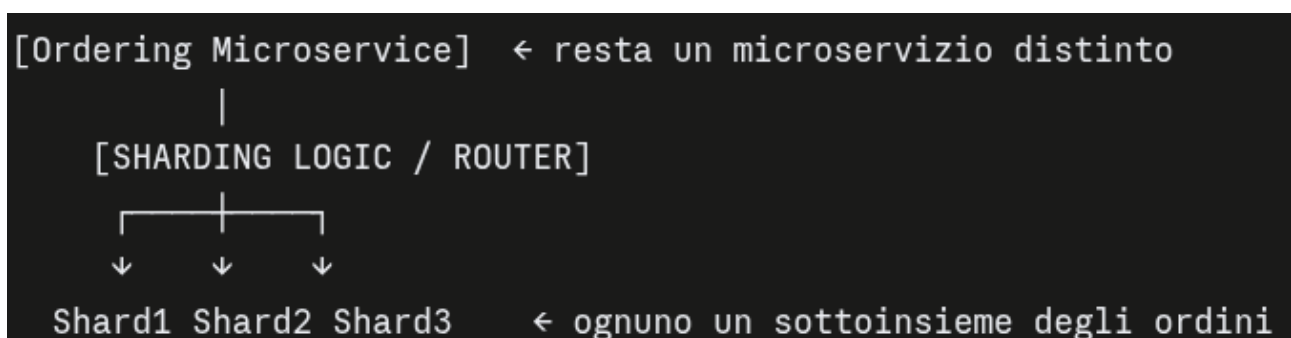
Toolkit aggiornato

Nuovi requisiti non funzionali:

- **Estrema scalabilità** per data store ad alto carico (es. Orders DB)
- **Alta disponibilità e fault tolerance a livello di dati shardati**
- **Mantenere le prestazioni** anche sotto carico massiccio

Pattern acquisiti in questa lezione: Database-per-Service, Polyglot Persistence, evitare lo Shared Database, comprensione di relazionali vs NoSQL, **teorema CAP**, strategie di partizionamento, **Database Sharding con Cassandra**.

Il design



*"La logica applicativa del microservizio Ordering rimane un microservizio distinto. Tuttavia, il database non è più una singola istanza. I dati degli ordini sono **partizionati orizzontalmente** su più shard."*

*"Ad esempio, si potrebbe shardare **in base all'intervallo di ID cliente, alla data dell'ordine, o a un hash dell'ID ordine**. Un livello logico di sharding è responsabile di indirizzare qualsiasi query o scrittura allo shard corretto in base alla partition key."*

Le 3 opzioni tecnologiche

Approccio	Note
Shardare un RDBMS (PostgreSQL, MySQL)	<i>"Richiede uno sforzo notevole. Gestire modifiche allo schema, riequilibrare gli shard e gestire query cross-shard può essere molto complesso"</i>
NoSQL nativamente distribuiti ★	<i>"Spesso un percorso più semplice. Molti sono progettati da zero con sharding, distribuzione e replica automatici incorporati. Apache Cassandra ne è un esempio lampante: peer-to-peer masterless, suddivide automaticamente i dati tra i nodi in base a una partition key. MongoDB supporta sharded cluster"</i>
Cloud-native	<i>Amazon DynamoDB, Azure Cosmos DB — "scalabilità massiccia con partizionamento automatico gestito dal cloud provider"</i> <i>"E sì, abbiamo imparato col teorema CAP che quando scegliamo database come Cassandra che danno priorità a disponibilità e tolleranza alle partizioni, stiamo spesso abbracciando l'eventual consistency — e questo va preso in considerazione nella progettazione."</i>

★ I 5 svantaggi dello sharding

1. Aumento della complessità architetturale

*"Bisogna **progettare la strategia, scegliere una shard key appropriata, implementare la logica di routing, e riequilibrare i dati se gli shard diventano disomogenei.**"*

2. Query e join cross-shard ★

*"Se devi eseguire query che coprono più shard — ad esempio "**ottieni tutti gli ordini di tutti i clienti effettuati ieri**" — queste possono diventare molto complesse e inefficienti. Spesso è necessario **progettare il modello dati con la denormalizzazione, o affidarsi a join a livello applicativo o a viste aggregate.**"*

3. Transazioni che si estendono su più shard

*"Implementare **transazioni ACID che coinvolgono dati su più shard è estremamente difficile e spesso non supportato direttamente. È qui che pattern come le Saga diventano necessari.**"*

4. Hotspot

*"Se la **shard key non è stata scelta bene, alcuni shard possono diventare "caldi"** — ricevendo una quantità sproporzionata di traffico mentre altri sono sottoutilizzati, annullando alcuni vantaggi della scalabilità."*

5. Costi operativi

*"Gestire un sistema di database distribuito con più shard **richiede maggiore impegno operativo.**"*

★ *"Lo sharding è una tecnica potente, ma non è un pranzo gratis. È tipicamente riservato ai servizi che richiedono davvero quel livello di scalabilità dei dati."*

Problem: Cross-Service Queries and Write Commands on Distributed Scaled Databases

Lo stato raggiunto

*"Abbiamo abbracciato: **Database-per-Service**, **polyglot persistence** per usare la tecnologia ottimale per ciascun servizio, e il **pattern di sharding** per servizi come Ordering che devono gestire volumi enormi. Abbiamo anche considerato il **teorema CAP** per capire i trade-off."*

"Questi sono eccellenti per la scalabilità e l'autonomia dei singoli servizi. Tuttavia, questi ambienti di dati altamente distribuiti e potenzialmente shardati comportano nuove domande e sfide."

Le 3 domande aperte

1. Come si eseguono le query cross-service?

"Come si eseguono le query che devono unire o aggregare dati ora sparsi in questi database indipendenti e potenzialmente shardati?"

È il problema del *"niente join"* del Database-per-Service, ora **aggravato** dallo sharding: prima non potevi fare join tra due database, ora **non li puoi fare nemmeno dentro lo stesso database**.

2. Come si gestiscono le scritture cross-service?

"Come si gestiscono le operazioni di scrittura o i comandi che potrebbero influire logicamente sui dati in più servizi, o che devono essere ottimizzati per scenari ad alto throughput?"

3. Si possono separare letture e scritture? ★

****Possiamo ottimizzare ulteriormente il sistema separando la gestione delle operazioni di LETTURA da quella delle operazioni di SCRITTURA, soprattutto in scala?"*

Questa è l'intuizione che porta direttamente a **CQRS**: letture e scritture hanno profili di carico, forme di dati e requisiti di coerenza **completamente diversi**. Perché usare lo stesso modello per entrambe?

La soluzione

*"Come soluzione, impareremo i **data & query pattern dei microservizi** e le **best practice**, tra cui:"*

- **Materialized View pattern**
- **CQRS pattern** (Command Query Responsibility Segregation)
- **Event Sourcing pattern**